Virtual Cosmos

2D Proximity-Based Virtual Environment with Real-Time Chat

A full-stack multiplayer application where users navigate a 2D cosmos, and chat rooms automatically connect and disconnect based on proximity — simulating real-world spatial interaction in a virtual space.

Technology: React • PixiJS • Socket.IO • Node.js • MongoDB

Type: Full-Stack Web Application

Date: April 2026

Table of Contents

1. Project Overview & Objectives
2. System Architecture
   High-level design, data flow, client-server model
3. Technology Stack & Justification
4. Project Structure & File Organization
5. Backend — Detailed Breakdown
   Server setup, Socket.IO events, proximity algorithm, MongoDB models
6. Frontend — Detailed Breakdown
   PixiJS canvas, game loop, avatar rendering, camera system, React components
7. Proximity Detection — Core Logic
   Algorithm, distance formula, connection lifecycle
8. Chat System — Design & Flow
   Room management, message persistence, auto connect/disconnect
9. Socket.IO Event Reference
10. Database Schema & Design
11. User Experience Flow
12. UI/UX Design Decisions
13. Setup & Running Instructions
14. Configuration Reference
15. Security Considerations

1. Project Overview & Objectives

1.1 What is Virtual Cosmos?

Virtual Cosmos is a **2D multiplayer virtual environment** where users can move around freely and interact with each other in real time. The core mechanic is **proximity-based chat**:

- **When two users come close** (within a 150-pixel radius) → a chat channel automatically connects between them
- **When users move apart** (beyond the radius) → the chat channel automatically disconnects

This simulates real-world spatial interaction — just like in a physical room, you can only talk to people who are near you. The experience is similar to platforms like *Gather.town* or *Cosmos.video*.

1.2 Objectives

The project demonstrates competency across four key engineering areas:

Area	What's Evaluated	How It's Addressed
Frontend Skills	Canvas rendering, movement, UI/UX quality	PixiJS 2D rendering engine, React component architecture, Tailwind styling, smooth 60fps game loop
Real-Time Logic	Proximity detection algorithm	Server-side Euclidean distance calculation with O(n) per-move checking, automatic room join/leave
Backend Fundamentals	User state, WebSocket management	Express + Socket.IO server, in-memory player state, MongoDB persistence for users and messages
System Design	Architecture, scalability thinking	Clean separation of concerns, event-driven architecture, stateless REST + stateful WebSocket hybrid

2. System Architecture

2.1 High-Level Architecture

The system follows a **client-server model** with WebSocket-based real-time communication:

```
┌──────────────────────────────────────────────────────────────────┐
│                        CLIENT (Browser)                          │
│                                                                  │
│  ┌──────────────┐  ┌──────────────┐  ┌──────────────────────┐  │
│  │ Join Screen  │  │   PixiJS     │  │    Chat Panel        │  │
│  │ (React)      │→ │   Canvas     │  │    (React)           │  │
│  │              │  │ (Game Loop)  │  │ - Auto connect       │  │
│  └──────────────┘  │ - Avatars    │  │ - Auto disconnect    │  │
│                    │ - Movement   │  │ - Message history    │  │
│                    │ - Camera     │  │ - Tabbed multi-chat  │  │
│                    └──────┬───────┘  └──────────┬───────────┘  │
│                           │                     │              │
│                    ┌──────┴─────────────────────┴──────┐       │
│                    │        Socket.IO Client           │       │
│                    └──────────────────┬────────────────┘       │
└──────────────────────────────────────┼────────────────────────┘
                                        │ WebSocket (ws://)
                                        │
┌──────────────────────────────────────┼────────────────────────┐
│                        SERVER (Node.js)│                       │
│                    ┌──────────────────┴────────────────┐      │
│                    │        Socket.IO Server           │      │
│                    └──────────────────┬────────────────┘      │
│                                       │                       │
│  ┌──────────────┐  ┌──────────────────┴──────────────────┐   │
│  │   Express    │  │          Game Logic Engine          │   │
│  │   REST API   │  │                                     │   │
│  │ /api/health  │  │ - Player state (Map)                │   │
│  │ /api/online  │  │ - Position tracking                 │   │
│  └──────────────┘  │ - Proximity detection algorithm     │   │
                     │ - Room management (Set)             │   │
```

```
│                    │  - Chat routing                        │   │
│                    └────────────────┬──────────────────────┘   │
│                                     │                          │
│                    ┌────────────────┴──────────────────────┐   │
│                    │           MongoDB (Mongoose)           │   │
│                    │  ┌──────────────┐  ┌─────────────────┐ │   │
│                    │  │ Users        │  │ Messages        │ │   │
│                    │  │ Collection   │  │ Collection      │ │   │
│                    │  └──────────────┘  └─────────────────┘ │   │
│                    └────────────────────────────────────────┘   │
└────────────────────────────────────────────────────────────────┘
```

2.2 Data Flow

There are three primary data flows in the system:

Flow 1: Position Synchronization

Client captures keyboard input → calculates new position → emits `player:move` to server → server updates in-memory state → broadcasts `player:moved` to all other clients → other clients update the PixiJS sprite position.

Flow 2: Proximity Detection

On every `player:move` event, the server iterates through all active players, calculates Euclidean distance. If distance < 150px and not already connected → joins both sockets to a shared room and emits `chat:connected`. If distance ≥ 150px and currently connected → removes from room and emits `chat:disconnected`.

Flow 3: Chat Messaging

When connected, client sends `chat:message` with room ID and content → server persists to MongoDB → broadcasts the message to all sockets in that room → both clients display the message in their chat panels.

3. Technology Stack & Justification

3.1 Frontend Technologies

Technology	Version	Purpose	Why This Choice
React	19.x	UI framework	Component-based architecture, excellent for managing UI state (chat panels, HUD). Hooks enable clean integration with PixiJS and Socket.IO lifecycles.
Vite	8.x	Build tool & dev server	Instant HMR for fast development. Native ES modules. Built-in proxy for API/WebSocket forwarding to backend during development.
PixiJS	7.x	2D canvas rendering	WebGL-accelerated 2D rendering. Hardware-accelerated graphics for smooth avatar rendering, particle effects, and large world with many players. Much more performant than DOM-based rendering for game-like interactions.
Tailwind CSS	4.x	UI styling	Utility-first approach enables rapid UI development. No context-switching between files. Consistent design system with minimal CSS footprint.
Socket.IO Client	4.x	WebSocket client	Matched with server-side Socket.IO. Automatic reconnection, fallback to polling, room abstraction built-in.

3.2 Backend Technologies

Technology	Version	Purpose	Why This Choice
Node.js	18+	Runtime	Event-driven, non-blocking I/O — ideal for real-time WebSocket applications handling many

			concurrent connections. Same language as frontend (JavaScript) reduces context switching.
Express	5.x	HTTP server	Lightweight, mature framework. Provides REST endpoints for health checks and online player queries. Pairs cleanly with Socket.IO.
Socket.IO	4.x	Real-time engine	Room-based architecture maps perfectly to proximity chat pairs. Built-in broadcast, room join/leave, automatic reconnection handling. Fallback to long-polling ensures reliability.
MongoDB	Atlas/Local	Database	Flexible schema for user profiles and chat messages. Document model suits our data (messages are self-contained documents). Mongoose ODM provides validation and type safety.
Mongoose	9.x	MongoDB ODM	Schema validation, middleware hooks, and clean query API. Upsert support simplifies user creation/update.

4. Project Structure & File Organization

```
virtual-cosmos/ ├── package.json # Root-level scripts (dev, install:all, build) ├── README.md # Project documentation ├── .gitignore # Git ignore rules │ ├── server/ # ── BACKEND ──────────────────────────── │ ├── index.js # Main server: Express + Socket.IO + game logic │ ├── package.json # Server dependencies │ ├── .env # Environment variables (PORT, MONGODB_URI) │ └── models/ │ ├── User.js # Mongoose schema: username, position, color │ └── Message.js # Mongoose schema: sender, content, roomId │ └── client/ # ── FRONTEND ──────────────────────────── ├── index.html # Entry HTML with Google Fonts ├── vite.config.js # Vite config with Tailwind + proxy ├── package.json # Client dependencies └── src/ ├── main.jsx # React DOM entry point ├── App.jsx # Root component (Join → Cosmos routing) ├── index.css # Tailwind imports + global styles ├── socket.js # Socket.IO client singleton ├── constants.js # Shared config (world size, speed, radius) └── components/ ├── JoinScreen.jsx # Username entry + avatar preview ├── Cosmos.jsx # Main canvas: PixiJS + game loop + sockets ├── ChatPanel.jsx # Auto-connecting chat with tabs └── HUD.jsx # Heads-up display overlay
```

4.1 Separation of Concerns

Layer	Files	Responsibility
Configuration	`constants.js` , `.env`	All tunable values (radius, speed, world size, ports) centralized
Network	`socket.js` , `server/index.js`	Socket.IO client/server setup, event handling
Game Engine	`Cosmos.jsx` (client), `index.js` (server)	Rendering, movement, proximity detection, position sync
Data	`models/User.js` , `models/Message.js`	Database schemas and persistence

UI	`JoinScreen`, `ChatPanel`, `HUD`	User interface components, styled with Tailwind

5. Backend — Detailed Breakdown

5.1 Server Setup (`server/index.js`)

The server creates an HTTP server with Express, then attaches Socket.IO to it:

```
const app = express();
const server = http.createServer(app);
const io = new Server(server, {
  cors: {
    origin: process.env.CLIENT_URL || 'http://localhost:5173',
    methods: ['GET', 'POST'],
  },
});
```

CORS is configured to accept connections from the Vite dev server. In production, this would be set to the deployed frontend URL.

5.2 In-Memory State Management

The server maintains two critical data structures in memory for real-time performance:

```
// Map: socketId → { userId, username, x, y, avatarColor }
const activePlayers = new Map();

// Set of active chat connections: "socketId1::socketId2" (sorted)
const activeConnections = new Set();
```

> **Why In-Memory?**
>
> Database queries on every position update (potentially 30 times/second/player) would create unacceptable latency. The in-memory Map provides O(1) lookups for player positions. MongoDB is used only for *persistence* (user profiles and chat history), not real-time state.

activePlayers Map Structure

Key	Type	Description
`socketId`	String (Map key)	Unique Socket.IO connection identifier
`userId`	String	Same as socketId (derived from connection)
`username`	String	Player's chosen display name
`x`	Number	Current X position in the 2D world
`y`	Number	Current Y position in the 2D world
`avatarColor`	String (hex)	Avatar color (e.g., "#6366f1")

activeConnections Set Structure

Each entry is a string in the format `"socketId1::socketId2"`, where the IDs are **alphabetically sorted**. This ensures a consistent key regardless of which player moved — `getRoomId("abc", "xyz")` and `getRoomId("xyz", "abc")` both return `"abc::xyz"`.

```
function getRoomId(id1, id2) {
  return [id1, id2].sort().join('::');
}
```

5.3 Player Lifecycle Events

player:join — User Enters the Cosmos

```
socket.on('player:join', async (data) => {
  const { username, avatarColor } = data;

  // Random spawn position within world bounds
  const x = 100 + Math.random() * 1800;
  const y = 100 + Math.random() * 800;

  // Store in memory
  activePlayers.set(socket.id, { userId: socket.id, username, x, y, avatarColor

  // Persist to MongoDB (upsert = create or update)
  await User.findOneAndUpdate(
    { username },
    { avatarColor, lastPosition: { x, y }, lastActive: new Date() },
    { upsert: true, new: true }
```

```
  );

  // Send current state to joining player
  socket.emit('player:current', Object.fromEntries(activePlayers));

  // Notify all other players
  socket.broadcast.emit('player:joined', { socketId: socket.id, ...playerData })

  // Check if spawned near anyone
  checkProximity(socket.id);
});
```

player:move — Position Update

```
socket.on('player:move', (data) => {
  const player = activePlayers.get(socket.id);
  if (!player) return;

  player.x = data.x;
  player.y = data.y;

  // Broadcast to everyone else
  socket.broadcast.emit('player:moved', { socketId: socket.id, x: data.x, y: dat

  // Re-evaluate all proximity relationships
  checkProximity(socket.id);
});
```

disconnect — User Leaves

When a player disconnects, the server:

1. Finds and removes all active chat connections involving this player
2. Notifies the other side of each connection with `chat:disconnected`
3. Saves the player's last position to MongoDB
4. Removes the player from the in-memory Map
5. Broadcasts `player:left` to all remaining players

5.4 REST API Endpoints

Endpoint	Method	Response	Purpose

`/api/health`	GET	`{ status: "ok", players: N }`	Health check, useful for monitoring & load balancers
`/api/online`	GET	Array of `{ socketId, username, avatarColor }`	List currently online players (useful for lobby UIs)

6. Frontend — Detailed Breakdown

6.1 Application Entry Flow

The React application manages two states:

```
// App.jsx — Root component
export default function App() {
  const [user, setUser] = useState(null);

  if (!user) return <JoinScreen onJoin={setUser} />;

  return <Cosmos username={user.username} avatarColor={user.avatarColor} />;
}
```

`user === null` → Show the Join Screen. Once the user submits a name → render the Cosmos.

6.2 JoinScreen Component

The entry screen provides:

- **Username input** — with 20-character limit and autofocus
- **Avatar preview** — shows the randomly assigned color with the user's first initial
- **Control hints** — informs users about WASD/Arrow key movement
- **Form validation** — submit button disabled until a non-empty username is entered

The avatar color is randomly assigned from a curated palette of 10 visually distinct colors defined in `constants.js`.

6.3 Cosmos Component — The Core Engine

This is the most complex component, integrating PixiJS rendering, keyboard input, Socket.IO events, and camera control. It runs within a single `useEffect` that manages the full lifecycle.

6.3.1 PixiJS Application Setup

```
const app = new PIXI.Application({
  width: window.innerWidth,
```

```
  height: window.innerHeight,
  backgroundColor: 0x0f0f1a,    // Dark cosmos background
  antialias: true,               // Smooth edges
  resolution: window.devicePixelRatio || 1,  // Retina support
  autoDensity: true,
});
```

The PIXI application renders to a WebGL canvas. A **world container** holds all game objects (grid, stars, avatars) and is moved to implement camera following.

6.3.2 World Layers (Rendering Order)

Layer	Z-Order	Content
Stars	Bottom	200 randomly placed semi-transparent white dots — creates a cosmic atmosphere
Grid	Middle	60px-spaced grid lines with subtle opacity — provides spatial reference and depth
Avatars	Top	Player circles with glow, initials, name labels, and (for local player) proximity ring

6.3.3 Avatar Rendering

Each avatar is a PixiJS Container with multiple layered elements:

```
Avatar Container
├── Proximity Ring (local player only)
│   ├── Filled circle: radius=150px, color=avatarColor, alpha=0.03
│   └── Dashed ring: 36 arc segments, alpha=0.25
├── Glow: circle, radius=30px, avatarColor, alpha=0.15
├── Main Circle: radius=22px, solid avatarColor, white border
├── Initial Letter: centered "A" in white, bold, 18px
└── Name Label: below avatar, 11px, white with drop shadow
```

6.3.4 Keyboard Input & Movement

Input uses a `Set` to track currently pressed keys, allowing simultaneous key presses (diagonal movement):

```
// In the game loop (ticker callback):
const keys = keysRef.current;
```

```
let dx = 0, dy = 0;

if (keys.has('w') || keys.has('arrowup'))    dy -= MOVE_SPEED;
if (keys.has('s') || keys.has('arrowdown'))  dy += MOVE_SPEED;
if (keys.has('a') || keys.has('arrowleft'))  dx -= MOVE_SPEED;
if (keys.has('d') || keys.has('arrowright')) dx += MOVE_SPEED;

// Diagonal normalization: prevents faster diagonal movement
if (dx !== 0 && dy !== 0) {
  const factor = Math.SQRT1_2;  // ≈ 0.707
  dx *= factor;
  dy *= factor;
}
```

Diagonal Normalization

Without normalization, moving diagonally (pressing W+D) would result in a speed of $\sqrt{(4^2 + 4^2)} \approx 5.66$px/frame — 41% faster than cardinal movement. Multiplying by $\sqrt{(1/2)} \approx 0.707$ brings diagonal speed back to exactly 4px/frame, ensuring uniform movement speed in all directions.

6.3.5 Camera System

```
// Smooth camera follow with lerp (linear interpolation)
const targetX = -localPosRef.current.x + window.innerWidth / 2;
const targetY = -localPosRef.current.y + window.innerHeight / 2;
world.x += (targetX - world.x) * 0.1;  // 10% lerp factor
world.y += (targetY - world.y) * 0.1;
```

The camera doesn't jump to the player position — it smoothly interpolates at 10% per frame, creating a fluid following effect. The world container itself is translated (the viewport stays fixed), so the local player always appears near the center of the screen.

6.3.6 Position Emit Throttling

```
// Throttle to ~30fps (33ms intervals)
const now = Date.now();
if (now - lastEmit > 33) {
  socket.emit('player:move', { x: newX, y: newY });
  lastEmit = now;
}
```

While the game loop runs at 60fps, network position updates are throttled to ~30fps. This reduces bandwidth by 50% while remaining visually smooth, since remote avatars are updated at each received position rather than interpolated.

6.4 ChatPanel Component

The chat panel provides:

- **Auto-selection** — automatically selects the first active chat when connections change
- **Tabbed interface** — when connected to multiple nearby users, tabs allow switching between conversations
- **Message bubbles** — own messages align right (indigo), others align left (dark gray)
- **Auto-scroll** — always scrolls to the latest message
- **Connection notice** — shows "Connected with [name]" when a chat begins
- **Timestamps** — each message shows HH:MM format
- **Minimize button** — can minimize the panel; a floating button shows active chat count

6.5 HUD Component

The heads-up display provides at-a-glance information:

- **Top-left:** Username with green "online" indicator
- **Top-center:** "VIRTUAL COSMOS" title with gradient
- **Top-right:** Online player count + active chat connections count
- **Bottom-left:** Control hints (WASD), proximity zone legend, current coordinates

All HUD elements use `pointer-events-none` on the container to prevent them from intercepting canvas clicks, with `pointer-events-auto` on interactive elements.

7. Proximity Detection — Core Logic

7.1 The Algorithm

Proximity detection is the **core mechanic** of Virtual Cosmos. It runs on the server to prevent cheating and ensure consistency. The algorithm executes every time any player moves.

```
const PROXIMITY_RADIUS = 150; // pixels

function getDistance(a, b) {
  return Math.sqrt((a.x - b.x) ** 2 + (a.y - b.y) ** 2);
}

function checkProximity(movedSocketId) {
  const movedPlayer = activePlayers.get(movedSocketId);
  if (!movedPlayer) return;

  for (const [otherSocketId, otherPlayer] of activePlayers) {
    if (otherSocketId === movedSocketId) continue;

    const dist = getDistance(movedPlayer, otherPlayer);
    const roomId = getRoomId(movedSocketId, otherSocketId);
    const isConnected = activeConnections.has(roomId);

    if (dist < PROXIMITY_RADIUS && !isConnected) {
      // ── CONNECT ──
      activeConnections.add(roomId);
      // Join both sockets to shared Socket.IO room
      io.sockets.sockets.get(movedSocketId)?.join(roomId);
      io.sockets.sockets.get(otherSocketId)?.join(roomId);
      // Load chat history from MongoDB and notify both clients
      // ... emit 'chat:connected' to both
    }
    else if (dist >= PROXIMITY_RADIUS && isConnected) {
      // ── DISCONNECT ──
      activeConnections.delete(roomId);
      // Remove both from the room
      io.sockets.sockets.get(movedSocketId)?.leave(roomId);
      io.sockets.sockets.get(otherSocketId)?.leave(roomId);
      // Notify both clients
      // ... emit 'chat:disconnected' to both
    }
```

```
    }
  }
```

7.2 Distance Formula

The Euclidean distance formula is used:

```
d = √( (x₂ − x₁)² + (y₂ − y₁)² )

Example:
Player A at (200, 300)
Player B at (300, 350)
d = √( (300−200)² + (350−300)² )
d = √( 10000 + 2500 )
d = √12500
d ≈ 111.8 pixels

111.8 < 150 → CONNECTED ✓
```

7.3 Connection State Machine

```
┌──────────────┐
│ DISCONNECTED │
│  (default)   │
└──────┬───────┘
       │
distance < 150px
       │
┌──────▼───────┐
│  CONNECTED   │ ←── Chat room created
│  (chatting)  │     Messages flow
└──────┬───────┘
```

```
                                        │
                                distance ≥ 150px
                              OR player disconnects
                                        │
                                   ┌────▼────┐
                        │ DISCONNECTED │ ←── Chat room destroyed
                         │  (default)   │     Chat panel hides
                                   └─────────┘
```

7.4 Complexity Analysis

Metric	Value	Notes
Per-move check	O(n)	Iterates through all other players. For n=100 players, that's 99 distance calculations per move.
Distance calc	O(1)	Two subtractions, two multiplications, one addition, one sqrt.
Connection lookup	O(1)	Set.has() on the roomId string — hash-based constant time.
Total per move	O(n)	Scales linearly with player count. Efficient for hundreds of players.

7.5 Visual Representation

On the client, the local player's proximity radius is visualized as:

- A **semi-transparent filled circle** (3% opacity) showing the detection zone
- A **dashed ring border** (25% opacity) at the exact radius boundary

This provides clear visual feedback so users understand when they're about to connect or disconnect.

8. Chat System — Design & Flow

8.1 Room-Based Architecture

Chat rooms are managed using Socket.IO's built-in room system. Each pair of connected players gets a unique room ID:

```
// Room ID = sorted socket IDs joined with "::"
// e.g., "abc123::xyz789"
const roomId = getRoomId(socketId1, socketId2);

// Join both players to the room
io.sockets.sockets.get(socketId1)?.join(roomId);
io.sockets.sockets.get(socketId2)?.join(roomId);

// Messages sent to the room reach both players
io.to(roomId).emit('chat:message', message);
```

8.2 Connection Lifecycle

1. **Proximity Detected (server-side)**
   Distance check triggers. Both socket IDs are joined to a shared room. The server loads the last 50 messages from MongoDB for this room ID.

2. **chat:connected emitted to both clients**
   Each client receives: peer's name, color, socket ID, room ID, and message history array. The ChatPanel component adds this to its `chats` Map state.

3. **Chat Panel Appears**
   React re-renders. The ChatPanel detects `chats.size > 0` and renders. If multiple chats exist, tabs appear for each connected peer.

4

Messaging

User types a message → client emits `chat:message { roomId, content }` → server persists to MongoDB → server broadcasts to the room → both clients display the message.

5

Proximity Lost (server-side)

Distance check triggers disconnect. Both sockets leave the room. `chat:disconnected` emitted. Client removes the peer from the `chats` Map. If no chats remain, the panel hides.

8.3 Message Persistence

```
// server/models/Message.js
const messageSchema = new mongoose.Schema({
  sender: String,           // Username of sender
  senderColor: String,      // Hex color for display
  content: String,          // Message text
  roomId: String,           // Indexed — for quick history lookup
  timestamp: { type: Date, default: Date.now },
});
```

When a chat connects, the server queries the last 50 messages for that room and sends them as history. This means if two users reconnect (walk away then return), their previous conversation is preserved.

8.4 Client-Side Chat State

```
// Cosmos.jsx state
const [chats, setChats] = useState(new Map());

// Map structure: peerId → {
//   peerName: "Alice",
//   peerColor: "#6366f1",
//   roomId: "socketA::socketB",
//   messages: [{ sender, senderColor, content, timestamp }, ...]
// }
```

Using a Map keyed by peer socket ID allows O(1) operations for adding, removing, and looking up chat conversations.

9. Socket.IO Event Reference

9.1 Client → Server Events

Event	Payload	When Emitted	Server Action
`player:join`	`{ username, avatarColor }`	After socket connects, immediately on Cosmos mount	Creates player in memory & DB, broadcasts to others, sends current state back
`player:move`	`{ x, y }`	Every ~33ms while user is moving (throttled)	Updates in-memory position, broadcasts to others, runs proximity check
`chat:message`	`{ roomId, content }`	When user sends a chat message	Persists to MongoDB, broadcasts to the room

9.2 Server → Client Events

Event	Payload	Recipient	Client Action
`player:current`	`{ [socketId]: playerData }`	Joining player only	Creates all existing avatars on canvas, sets own position
`player:joined`	`{ socketId, username, avatarColor, x, y }`	All except joiner	Creates new avatar sprite on canvas
`player:moved`	`{ socketId, x, y }`	All except mover	Updates avatar position on canvas
`player:left`	`{ socketId }`	All remaining	Removes avatar from canvas,

			decrements player count
`chat:connected`	`{ peerId, peerName, peerColor, roomId, history[] }`	Both players in pair	Adds peer to chats Map, shows ChatPanel with history
`chat:disconnected`	`{ peerId, roomId }`	Both players in pair	Removes peer from chats Map, hides ChatPanel if empty
`chat:message`	`{ sender, senderColor, content, roomId, timestamp }`	All in room	Appends message to the correct chat's messages array

10. Database Schema & Design

10.1 Users Collection

```
// server/models/User.js
{
  _id: ObjectId,
  username: { type: String, required: true, unique: true, trim: true },
  avatarColor: { type: String, default: '#6366f1' },
  lastPosition: {
    x: { type: Number, default: 400 },
    y: { type: Number, default: 300 }
  },
  lastActive: { type: Date, default: Date.now },
  createdAt: Date,   // auto (timestamps: true)
  updatedAt: Date    // auto (timestamps: true)
}
```

Purpose: Persists user profiles across sessions. Tracks the last known position and activity time. Uses upsert on join, so returning users are updated rather than duplicated.

10.2 Messages Collection

```
// server/models/Message.js
{
  _id: ObjectId,
  sender: { type: String, required: true },
  senderColor: { type: String, default: '#6366f1' },
  content: { type: String, required: true },
  roomId: { type: String, required: true, index: true },
  timestamp: { type: Date, default: Date.now }
}
```

Purpose: Persists all chat messages. The `roomId` field is indexed for fast history lookups when users reconnect. Messages are queried with `.sort({ timestamp: -1 }).limit(50)` and reversed for chronological order.

10.3 Graceful Degradation

MongoDB is Optional

The server catches MongoDB connection errors and logs a warning. If MongoDB is unavailable, the app still works fully — movement, proximity detection, and real-time chat all function. Only message persistence and user profile storage are affected.

11. User Experience Flow

1. **Open the Application**

   User navigates to `http://localhost:5173`. The Join Screen loads with a dark cosmos theme.

2. **Enter Username**

   User types their name. An avatar preview updates in real-time showing their assigned color and initial. They click "Enter Cosmos".

3. **Spawn in the Cosmos**

   Socket connects. User appears at a random position in the 2000×1000 world. They see the star field, grid, their avatar with a proximity ring, and the HUD showing stats.

4. **See Other Users**

   All online players are visible as colored circles with names. Their positions update in real-time as they move. The player count in the HUD reflects current occupancy.

5. **Move Around**

   Using WASD or Arrow keys, the user navigates the cosmos. The camera smoothly follows. Movement is clamped to world bounds. The position coordinates update in the HUD.

6. **Approach Another User → Chat Connects**

   When the distance to another player drops below 150px, the chat panel slides in from the right. It shows "Connected with [name]" and loads any previous message history. The connections count in the HUD increments.

7

Chat in Real-Time

Users can type messages. Messages appear as styled bubbles — own messages on the right (indigo), peer messages on the left (gray). All messages have timestamps.

8

Move Away → Chat Disconnects

When the user moves beyond 150px from the peer, the chat connection is terminated. The peer's tab disappears from the chat panel. If no connections remain, the chat panel hides entirely.

9

Multiple Connections

If the user is near multiple people simultaneously, the chat panel shows tabs for each connection, allowing switching between conversations.

12. UI/UX Design Decisions

12.1 Visual Theme

Element	Value	Reasoning
Background	`#0f0f1a` (near-black with blue tint)	Creates a "space" atmosphere befitting the cosmos theme
Panels	`#1a1a2e` with `backdrop-blur`	Glassmorphism effect — visible but non-obstructive
Borders	`#2a2a4a`	Subtle separation without harsh contrast
Primary	Indigo (`#6366f1`)	Modern, professional, excellent contrast on dark backgrounds
Font	Inter	Clean, geometric sans-serif optimized for screen readability

Corners	`rounded-xl` / `rounded-2xl`	Soft, friendly, modern aesthetic

12.2 Key UX Principles

- **Minimal friction:** One input (username) to enter. No registration or login required.
- **Clear affordances:** Control hints always visible. Proximity ring shows detection zone.
- **Non-intrusive HUD:** Stats visible but don't obstruct gameplay. `pointer-events-none` prevents accidental clicks.
- **Automatic behavior:** Chat connects/disconnects automatically — no manual action needed.
- **Visual feedback:** Green pulse dots for "online" status. Animated elements for active connections.
- **Responsive:** Canvas and UI resize with the window. Panels use fixed positioning.

13. Setup & Running Instructions

13.1 Prerequisites

- **Node.js** version 18 or higher
- **npm** (included with Node.js)
- **MongoDB** — local installation or MongoDB Atlas cloud (optional — app works without it)

13.2 Installation

```
# Clone the repository
git clone <repository-url>
cd virtual-cosmos

# Install all dependencies (server + client)
npm run install:all
```

13.3 Configuration

Edit `server/.env` to configure:

```
PORT=3001                              # Server port
MONGODB_URI=mongodb://localhost:27017/virtual-cosmos  # MongoDB URI
CLIENT_URL=http://localhost:5173       # Frontend URL for CORS
```

13.4 Running in Development

```
# Option A: Two terminals
# Terminal 1 - Start the backend
cd server && npm run dev

# Terminal 2 - Start the frontend
cd client && npm run dev
```

```
# Option B: Single command (from root)
npm run dev
```

13.5 Testing the Application

1. Open **http://localhost:5173** in your browser
2. Enter a username (e.g., "Alice") and click "Enter Cosmos"
3. Open another browser tab (or use incognito/different browser)
4. Enter a different username (e.g., "Bob")
5. Use WASD/Arrow keys to move both avatars close together
6. When they're within 150px — the chat panel appears on both screens
7. Send messages back and forth
8. Move one avatar away — the chat panel disappears

13.6 Building for Production

```
# Build the frontend
npm run build

# The output is in client/dist/ — serve with any static file server
# The backend runs as-is with: cd server && npm start
```

14. Configuration Reference

14.1 Client Configuration (`client/src/constants.js`)

Constant	Default	Unit	Description
`WORLD_WIDTH`	2000	px	Total width of the explorable cosmos
`WORLD_HEIGHT`	1000	px	Total height of the explorable cosmos
`MOVE_SPEED`	4	px/frame	Player movement speed (at 60fps = 240px/sec)

`PROXIMITY_RADIUS`	150	px	Visual display radius on client
`AVATAR_RADIUS`	22	px	Size of the avatar circle

14.2 Server Configuration (`server/index.js`)

Variable	Default	Description
`PROXIMITY_RADIUS`	150	Server-side detection radius (must match client)
`PORT`	3001	Server listening port (from .env)
`MONGODB_URI`	mongodb://localhost:27017/virtual-cosmos	Database connection string

Important: Radius Sync

The `PROXIMITY_RADIUS` value must be the same on both client and server. The client uses it for the visual ring; the server uses it for the actual detection. If they differ, the visual feedback will not match the actual connection behavior.

15. Security Considerations

Concern	How Addressed
Server-side proximity	Proximity detection runs on the server, not the client. Users cannot fake being near someone to force a chat connection.
CORS	Socket.IO and Express are configured to only accept connections from the specified client URL.
Input validation	Chat messages are trimmed and checked for empty content before being saved or broadcast. Username is trimmed and limited to 20 characters on the client.

Room isolation	Chat messages are only sent to the specific Socket.IO room. A player cannot receive messages from rooms they haven't been added to by the server.
Position bounds	Client-side position clamping prevents users from moving outside the world. Server could add server-side validation for additional security.
Environment variables	Database credentials and configuration stored in .env, excluded from git via .gitignore.

Virtual Cosmos — 2D Proximity-Based Virtual Environment

Built with React + PixiJS + Socket.IO + Node.js + MongoDB

Document generated April 2026